

Factors and variable elimination

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	Factors versus kernels	2
4	The factor algebra	2
5	The asia network as a factor graph	4
6	Variable elimination versus brute force	5
7	Evidence	6
8	Ancestral sampling as a cross-check	6
9	From a model file to a posterior	7
10	Summary	9
11	References	9

1 Overview

`BayesianNetworkInference.jl` answers queries such as $P(\text{dysp} \mid \text{asia} = \text{yes})$ on a Bayesian network by variable elimination: the network is turned into a bag of *factors*, evidence is applied by slicing, the variables outside the query are summed out one at a time, and the product of what remains is normalised. This vignette covers the factor algebra and the elimination step, checked throughout against a brute-force joint table. The companion vignette looks at elimination orderings.

Variable elimination is bucket elimination ([Dechter 1999](#)) in the factor presentation of Koller and Friedman (2009, ch. 9); summing out one variable at a time, and touching only the factors that mention it, is the optimisation of Zhang and Poole (1994).

2 Setup

```
using BayesianNetworkInference
using FiniteKernels
```

3 Factors versus kernels

`FiniteKernels.jl` represents a conditional probability table as a `FiniteKernel`, a morphism with a fixed input/output partition and a normalisation invariant. Inference works instead with `Factors`: unnormalised tensors over an ordered scope, closed under pointwise product and summation. The two types are distinct on purpose, and the conversion is explicit.

```
rain = FiniteAxis(:Rain, [:yes, :no])
grass = FiniteAxis(:Grass, [:wet, :dry])

p_rain = cpt(rain, [0.2, 0.8])           # kernel I -> Rain
p_grass = cpt(rain, grass, [0.9 0.1; 0.2 0.8]) # kernel Rain -> Grass

f_rain = Factor(p_rain, :Rain)           # scope (Rain,)
f_grass = Factor(p_grass, [:Rain], :Grass) # scope (Rain, Grass)
f_grass
```

`Factor{Float64}` over `(:Rain, :Grass)` with size `(2, 2)`

The factor of a mechanism has scope `(parents..., child)` and its table is the parents-first layout `cpt(k)`, so `f_grass.table[i, j] = P(Grass = j | Rain = i)`:

```
f_grass.table
```

```
2×2 Matrix{Float64}:
 0.9  0.1
 0.2  0.8
```

4 The factor algebra

`multiply` aligns scopes automatically (the result's scope is the first factor's scope followed by the new variables of the second), `marginalize` sums variables out, `condition` slices at observed labels, `normalize` rescales to unit mass and `reorder` permutes the scope.

```
joint = multiply(f_rain, f_grass)      # P(Rain, Grass)
scope(joint), joint.table
```

```
([:Rain, :Grass], [0.18000000000000002 0.020000000000000004; 0.16000000000000003
0.64000000000000001])
```

```
marginalize(joint, :Rain).table      # P(Grass) = [0.34, 0.66]
```

```
2-element Vector{Float64}:
 0.34000000000000001
 0.66000000000000001
```

Multiplication is commutative *modulo axis order*: $f * g$ and $g * f$ have permuted scopes but are equal as factors.

```
multiply(f_grass, f_rain) ≈ joint, scope(multiply(f_grass, f_rain))
```

```
(true, [:Rain, :Grass])
```

Conditioning drops the observed axis; normalising the result gives a posterior.

```
normalize(condition(joint, Dict{:Grass => :wet})).table  # P(Rain | Grass = wet)
```

```
2-element Vector{Float64}:
 0.5294117647058822
 0.47058823529411764
```

A normalised factor with a chosen input/output split converts back to a kernel; the conversion checks normalisation and throws `KernelNormalizationError` otherwise.

```
FiniteKernel(f_grass, [:Rain], :Grass) ≈ p_grass
```

```
true
```

```
try
  FiniteKernel(joint, [:Rain], :Grass)
catch e
  e
end
```

```
KernelNormalizationError("kernel FiniteSpace(Rain{yes,no}) →
FiniteSpace(Grass{wet,dry}) is not normalised over its output axes", 0.8, 1.0e-8,
nothing)
```

5 The asia network as a factor graph

The classic *asia* network (Lauritzen and Spiegelhalter 1988) has eight binary variables. We build its kernels from the published tables and compile them to factors, one per mechanism; the `FactorGraph` records the variables' axes and, in provenance, where each factor came from.

```
yn = [:yes, :no]
ax(v) = FiniteAxis(v, yn)

either = zeros(2, 2, 2) # (lung, tub, either): either = lung OR tub
either[1, 1, :] = [1.0, 0.0]; either[2, 1, :] = [1.0, 0.0]
either[1, 2, :] = [1.0, 0.0]; either[2, 2, :] = [0.0, 1.0]
dysp = zeros(2, 2, 2) # (bronc, either, dysp)
dysp[1, 1, :] = [0.9, 0.1]; dysp[2, 1, :] = [0.7, 0.3]
dysp[1, 2, :] = [0.8, 0.2]; dysp[2, 2, :] = [0.1, 0.9]

kernels = Pair{Symbol,FiniteKernel}[
    :asia => cpt(ax(:asia), [0.01, 0.99]),
    :tub => cpt(ax(:asia), ax(:tub), [0.05 0.95; 0.01 0.99]),
    :smoke => cpt(ax(:smoke), [0.5, 0.5]),
    :lung => cpt(ax(:smoke), ax(:lung), [0.1 0.9; 0.01 0.99]),
    :bronc => cpt(ax(:smoke), ax(:bronc), [0.6 0.4; 0.3 0.7]),
    :either => cpt([ax(:lung), ax(:tub)], ax(:either), either),
    :xray => cpt(ax(:either), ax(:xray), [0.98 0.02; 0.05 0.95]),
    :dysp => cpt([ax(:bronc), ax(:either)], ax(:dysp), dysp)]
parents = Dict{Symbol,Symbol}[:asia => Symbol[], :tub => [:asia], :smoke => Symbol[], :lung =>
[:smoke],
    :bronc => [:smoke], :either => [:lung, :tub], :xray => [:either],
    :dysp => [:bronc, :either]]

factors = [Factor(k, parents[v], v) for (v, k) in kernels]
fg = FactorGraph(factors; provenance=first.(kernels))
```

`FactorGraph{Float64}` with 8 factors over 8 variables

```
variables(fg)
```

8-element Vector{Symbol}:

```
:asia
:tub
:smoke
:lung
:bronc
:either
:xray
:dysp
```

The interaction graph joins every pair of variables sharing a factor. Because each factor spans a child and all of its parents, it is the moral graph of the network.

```
using Graphs
g, vars, index = interaction_graph(fg)
[(vars[src(e)], vars[dst(e)]) for e in edges(g)]
```

```
10-element Vector{Tuple{Symbol, Symbol}}:
 (:asia, :tub)
 (:tub, :lung)
 (:tub, :either)
 (:smoke, :lung)
 (:smoke, :bronc)
 (:lung, :either)
 (:bronc, :either)
 (:bronc, :dysp)
 (:either, :xray)
 (:either, :dysp)
```

6 Variable elimination versus brute force

`joint_factor` multiplies every factor into the full 2^8 table (the oracle) and `brute_force_marginal` conditions and sums it. `variable_elimination` never builds that table; it also returns diagnostics about the elimination it performed.

```
p_dysp, diag = variable_elimination(fg, [:dysp])
p_dysp.table
```

```
2-element Vector{Float64}:
 0.4359706
 0.5640294
```

$P(\text{dysp} = \text{yes}) = 0.4360$ is the value published for asia and independently computed by `BayesianNetworkFormats.jl` from the file formats, which pins down the (parents..., child) axis convention.

```
p_dysp ≈ brute_force_marginal(fg, [:dysp])
```

```
true
```

```
diag
```

```
InferenceDiagnostics([:asia, :tub, :xray, :lung, :smoke, :either, :bronc], 8, 7, 2, false)
```

The largest intermediate factor had `diag.max_factor_size` entries and the induced width of the order was `diag.treewidth`; compare with the 256 entries of the joint.

7 Evidence

Evidence is a dictionary from variables to labels. Factors touching evidence are sliced before elimination, so the evidence variables never appear in the order.

```
ev = Dict(:asia => :yes, :xray => :yes)
post_ev, diag_ev = variable_elimination(fg, [:dysp]; evidence=ev)
post_ev.table, diag_ev.order
```

```
([0.6811011940658546, 0.31889880593414544], [:tub, :smoke, :lung, :either, :bronc])
```

```
post_ev ≈ brute_force_marginal(fg, [:dysp]; evidence=ev)
```

true

A joint query over several variables returns a factor in the requested order, and an empty query returns the probability of the evidence itself.

```
joint_q, _ = variable_elimination(fg, [:lung, :bronc]; evidence=Dict(:smoke =>
:yes))
scope(joint_q), joint_q.table
```

```
([:lung, :bronc], [0.05999999999999999 0.04; 0.54 0.36000000000000004])
```

```
p_evidence, _ = variable_elimination(fg, Symbol[]; evidence=ev)
p_evidence.table[]
```

0.0014509250000000003

`infer` is the generic entry point (it also accepts a `BayesModel` from `BayesianNetworks.jl`, see below); the backend chooses the algorithm and its ordering strategy.

```
infer(fg, :dysp; evidence=ev, backend=VariableElimination(; order=MinDegree()))[1]
≈ post_ev
```

true

8 Ancestral sampling as a cross-check

`ancestral_sample` draws joint samples by visiting the variables in topological order and sampling each from its kernel given its parents. Empirical marginals converge to the exact ones.

```
using Random
order = [:asia, :tub, :smoke, :lung, :bronc, :either, :xray, :dysp]
samples = ancestral_sample(kernels, order, parents, 20_000; rng=MersenneTwister(1))
empirical_marginal(samples, :dysp).table
```

```
2-element Vector{Float64}:
 0.43405
 0.56595
```

```
isapprox(empirical_marginal(samples, :dysp), p_dysp; atol=0.02)
```

```
true
```

9 From a model file to a posterior

The factors above were assembled by hand. `BayesianNetworks.jl` builds the same kind of network as a `BayesModel` (structure as an `ACSet`, kernels bound to mechanisms), reads it from the file formats of `BayesianNetworkFormats.jl`, and evaluates it by brute force. `compile` turns a model into a `FactorGraph`, one factor per mechanism with scope (`parents...`, `child`), and `infer(model, query)` does that and eliminates. Here is the SPEC section 45 reference habitat network read from its Netica file.

```
using BayesianNetworks
m = read_bayesnet(fixture_path("dne/habitat_reference.dne"))
```

```
BayesModel(7 variables, 7 mechanisms, 7 kernels)
```

```
fg_m = compile(m)
```

```
FactorGraph{Float64} with 7 factors over 7 variables
```

Every factor records the variable and the mechanism it came from.

```
fg_m.provenance
```

```
7-element Vector{Any}:
 (variable = :Climate, mechanism = :Climate_mechanism, id = 1)
 (variable = :Irrigation, mechanism = :Irrigation_mechanism, id = 2)
 (variable = :SoilMoisture, mechanism = :SoilMoisture_mechanism, id = 3)
 (variable = :GrazingPressure, mechanism = :GrazingPressure_mechanism, id = 4)
 (variable = :Vegetation, mechanism = :Vegetation_mechanism, id = 5)
 (variable = :HabitatQuality, mechanism = :HabitatQuality_mechanism, id = 6)
 (variable = :Occupancy, mechanism = :Occupancy_mechanism, id = 7)
```

```
post, diag_m = infer(m, :Occupancy)
post.table
```

```
2-element Vector{Float64}:
 0.5237590625
 0.4762409375
```

The oracle at this level is `marginal` from `BayesianNetworks.jl`, which enumerates the joint table of the model; the two agree to floating-point precision.

```
maximum(abs, post.table .- marginal(m, :Occupancy).table)
```

```
1.1102230246251565e-16
```

```
post ≈ Factor(marginal(m, :Occupancy), :Occupancy)
```

```
true
```

Evidence can be given explicitly or recorded on the model with `observe`; `posterior` returns a dictionary keyed by state.

```
posterior(m, :Occupancy; evidence=Dict(:Vegetation ⇒ :dense))
```

```
Dict{Symbol, Float64} with 2 entries:  
 :present => 0.6675  
 :absent  => 0.3325
```

```
posterior(observe(m, :Vegetation ⇒ :dense), :Occupancy)
```

```
Dict{Symbol, Float64} with 2 entries:  
 :present => 0.6675  
 :absent  => 0.3325
```

An intervention is a rewrite of the model's syntax (the mechanism of the intervened variable becomes a point mass), so the compiled factor graph changes and `infer` answers the interventional query without any special casing.

```
m_do = do_intervention(m, :GrazingPressure ⇒ :low)  
post_do, _ = infer(m_do, :Occupancy)  
post_do.table
```

```
2-element Vector{Float64}:  
 0.48833975  
 0.51166025
```

```
post_do ≈ Factor(marginal(m_do, :Occupancy), :Occupancy)
```

```
true
```

Seeing dense vegetation says something about the climate upstream; forcing dense vegetation does not.

```
posterior(observe(m, :Vegetation ⇒ :dense), :Climate)
```



```
Dict{Symbol, Float64} with 3 entries:  
:normal => 0.524995  
:wet    => 0.276395  
:dry    => 0.19861
```

```
posterior(do_intervention(m, :Vegetation => :dense), :Climate)
```

```
Dict{Symbol, Float64} with 3 entries:  
:normal => 0.5  
:wet    => 0.2  
:dry    => 0.3
```

The same network stored as a GeNIe .xdsl file gives identical numbers.

```
mx = read_bayesnet(fixture_path("xdsl/habitat_reference.xdsl"))  
infer(mx, :Occupancy)[1] ≈ post,  
infer(do_intervention(mx, :GrazingPressure => :low), :Occupancy)[1] ≈ post_do
```

```
(true, true)
```

10 Summary

Factors are unnormalised tensors over an ordered scope, closed under product, summation and slicing, and a Bayesian network is just a bag of them; variable elimination answers a query by conditioning, summing out the non-query variables one at a time and normalising, and it agreed with the brute-force joint on every example here. The cost of an elimination is set entirely by the order in which the variables are removed, which is what the next vignette, *Elimination orderings with CliqueTrees*, takes up.

11 References

- Dechter, Rina. 1999. “Bucket Elimination: A Unifying Framework for Reasoning.” *Artificial Intelligence* 113 (1–2): 41–85. [https://doi.org/10.1016/S0004-3702\(99\)00059-4](https://doi.org/10.1016/S0004-3702(99)00059-4).
- Koller, Daphne, and Nir Friedman. 2009. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- Lauritzen, Steffen L., and David J. Spiegelhalter. 1988. “Local Computations with Probabilities on Graphical Structures and Their Application to Expert Systems.” *Journal of the Royal Statistical Society, Series B* 50 (2): 157–224. <https://doi.org/10.1111/j.2517-6161.1988.tb01721.x>.
- Zhang, Nevin Lianwen, and David Poole. 1994. “A Simple Approach to Bayesian Network Computations.” *Proceedings of the Tenth Canadian Conference on Artificial Intelligence*, 171–78.